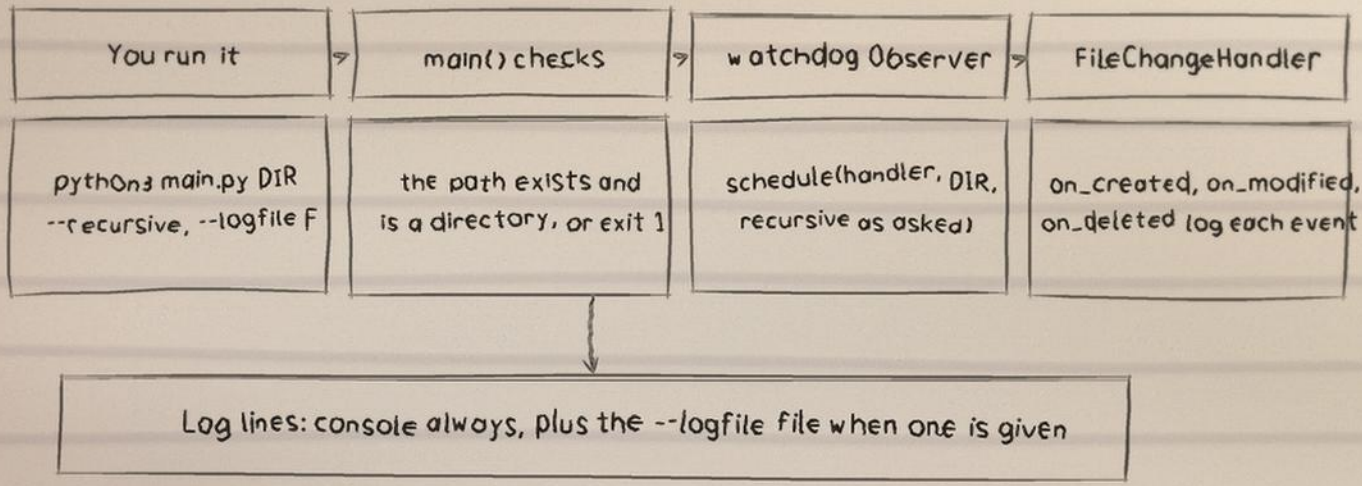
monitor-filechanges: a watcher in four parts

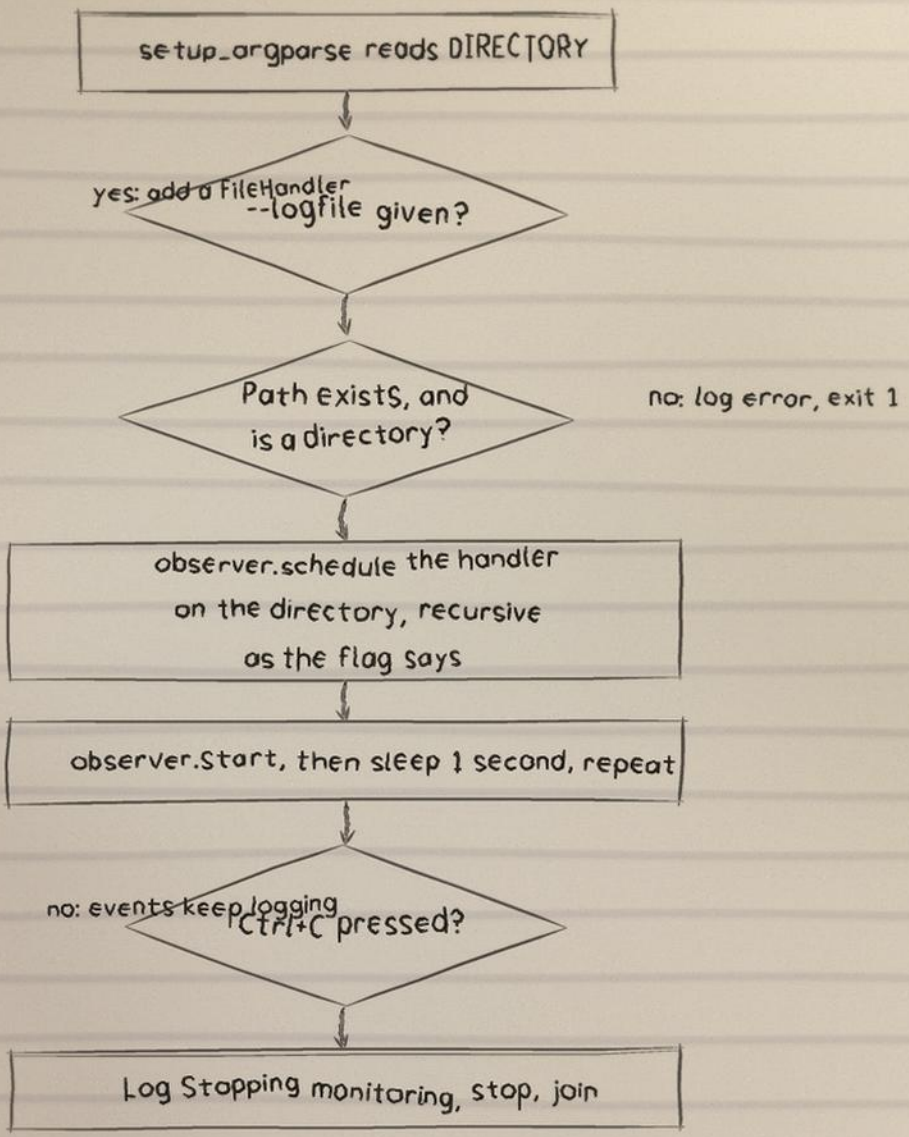


It runs in the foreground: a one second sleep loop until Ctrl+C, then observer.stop and join.

File content is never read. Only the event kind and the path are logged.

watch tag: GK

From start to Ctrl+C



Events arrive the whole time the loop sleeps: created, modified, deleted.

Product map: what changes, written down as they happen

CREATED	MODIFIED	DELETED
File created: PATH or Directory created: PATH, from on_created	File modified: PATH or Directory modified: PATH, from on_modified	File deleted: PATH or Directory deleted: PATH, from on_deleted

WHO IT SERVES	INPUTS	OUTPUTS	BOUNDARIES
Me, watching a folder while something else writes into it.	One directory path. Two flags: --recursive and --logfile.	Timestamped log lines, console always, a file too when asked.	No move or rename handling. Content is never read or stored.

One honest quirk from the README: creating a file usually logs a modify event for the file and its folder too.

GK

Sequence: one file appears in the watched folder

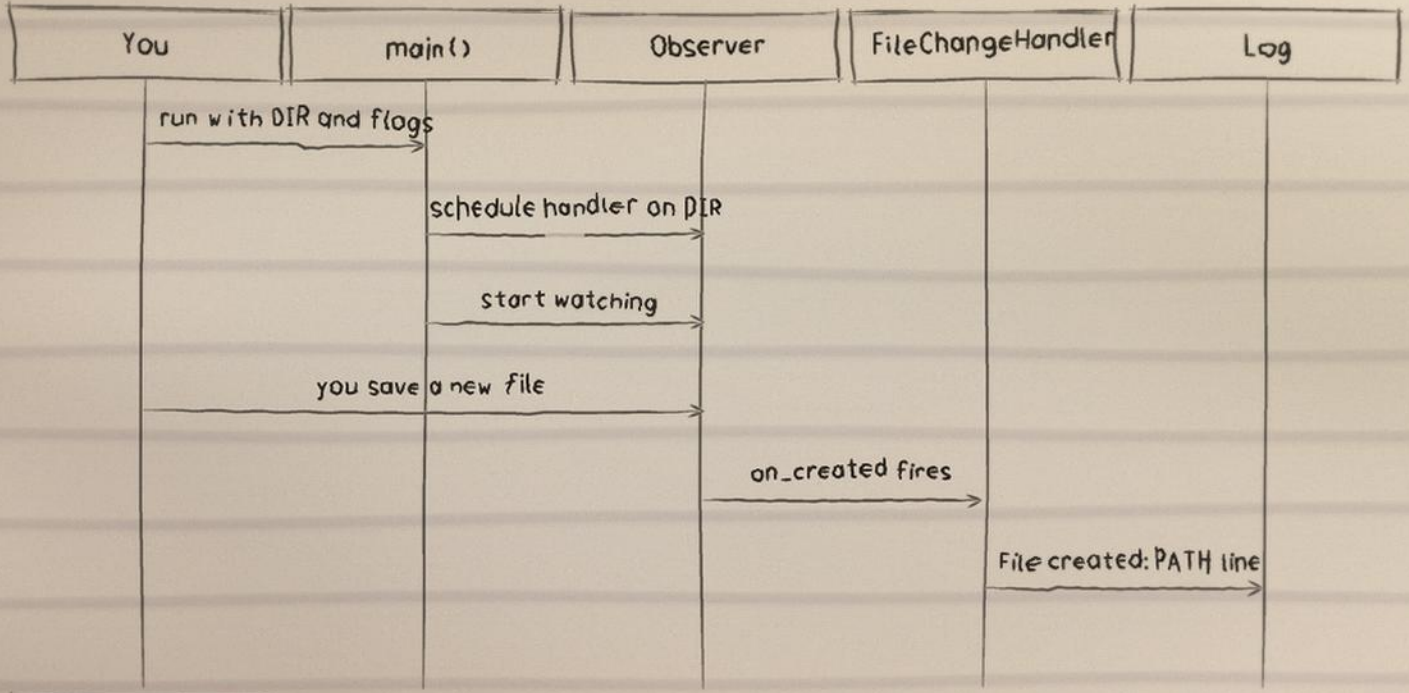
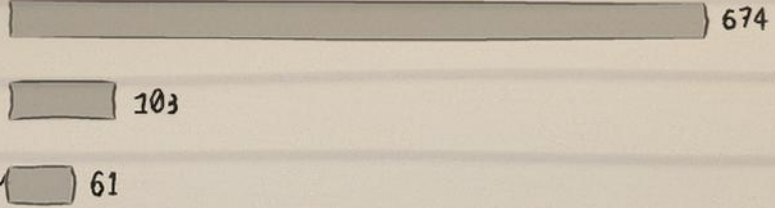


plate: Modify and delete follow the same path, through on_modified and on_deleted.

monitor-filechanges in numbers, counted 2026-10-30

Lines by kind, from git ls-files and a line count over each tracked text file.



The whole repo is 6 files and 7 commits. First commit 2025-01-02, last one 2026-10-30, both from git log. Tests: no test files and no test suite in this repo, so no pass is claimed. The watchdog package is not installed in my sandbox and installs are out of scope here, so the monitor itself was not run either.

counted and drawn by hand

Monitor Filechanges: schematic of the apparatus

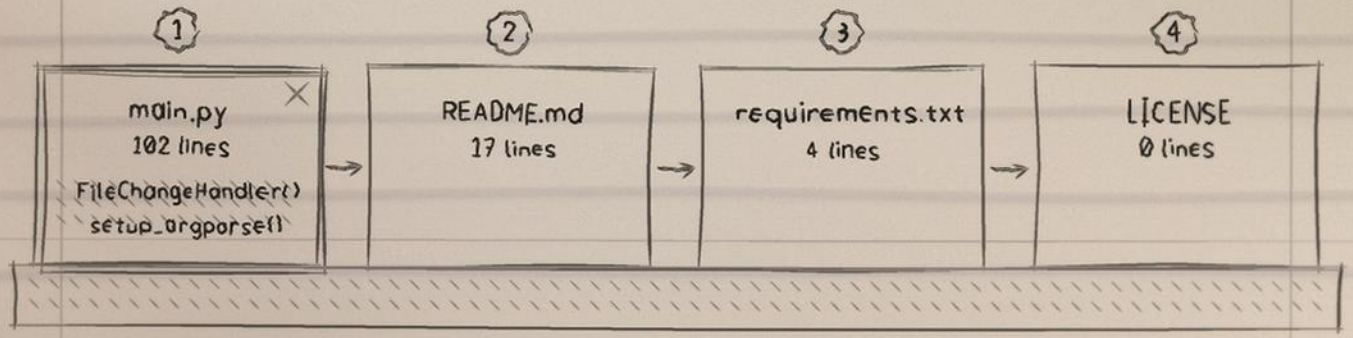
the system as a machine: parts, flow, and measurements

construction lines left in, as drawn

parts list

1. main.py
2. README.md
3. requirements.txt
4. LICENSE

in flow order, first to last



base plate: the repo itself, sectioned

span: 4 files, 123 lines counted

turns of the crank so far: 5 commits, 2025-01-02 to 2025-01-02

measured and sketched, GK